

邻舍 (Lin She) 校园二手交易微信小程序

开发文档

基于微信小程序原生框架与校园二手交易 Demo 业务流程

日期: 2026 年 6 月 26 日

项目项	内容
项目类型	微信小程序原生开发课程 Demo
技术栈	WXML / WXSS / JavaScript / 微信小程序 Page 框架
设计来源	页面原型 test2 高保真设计与 test2.1 Make 项目
页面规模	17 个页面，包含 5 个 TabBar 主入口与 12 个二级页面
质量状态	npm run check 已通过，页面四件套、Schema、页面原型 覆盖均已校验
演示说明	DEMO_GUIDE.md 记录开发者工具打开方式与主流程演示步骤

文档摘要

本文件是对“邻舍”校园二手交易小程序的开发初版说明，面向移动应用开发期末作业提交场景编写。文档从项目目标、需求范围、系统结构、核心业务实现、运行界面、代码证据和测试结果几个角度重新组织，不沿用旧文档的章节表述。

需要特别说明的是，本项目按课程 Demo 定位实现完整功能结构。学生认证、发布阻断、出价、结算、订单核销、消息聊天等流程均可在本地前端跑通；其中支付和核销是模拟流程，点击确认支付后自动进入待面交状态，点击确认收货后自动完成核销，不涉及真实金钱交易。

一、项目定位与开发范围

邻舍面向同校区学生之间的二手物品流转，核心场景不是跨城物流，而是“楼下见面、当场验货、即时确认”的校园近距离交易。产品设计把校园身份、面交地点和交易确认放在同等重要的位置，避免普通二手平台在校园场景中常见的信息分散与信任不足问题。

目标	实现方式	当前状态
校园二手交易	首页商品流、搜索筛选、商品详情、卖家主页	已实现
学生准入机制	auth 服务校验 .edu 邮箱，未认证阻断发布与支付	已实现为 Demo
线下面交闭环	结算页选择面交点，订单页展示地点与核销入口	已实现
交易安全表达	Demo 担保支付状态机 wait_pay -> wait_meetup -> completed	本地模拟
设计稿协作	读取 页面原型 test2/test2.1，记录 18 个 Screen 节点	已对齐

- 项目围绕校园二手交易场景设计核心路由，并增加搜索、出价、结算、钱包、安全中心、收藏、卖家主页、聊天详情等扩展页面。
- 项目采用微信原生框架，未引入跨端框架，便于在微信开发者工具中直接打开、编译和预览。
- 业务层使用 services 目录统一封装，避免把认证、订单、消息逻辑全部写死在页面事件中。

二、设计与工程协作流程

本项目采用先完成页面原型与功能拆分，再进行微信小程序原生页面开发和开发者工具运行验证的流程。开发过程中重点保证页面结构、数据渲染、交互跳转和本地 Demo 业务状态的一致性。

协作对象	输入内容	输出结果
页面原型	18 屏设计、品牌色、页面层次和组件布局	页面拆分、视觉风格、交互入口
开发实现	项目需求、页面原型资料、微信官方文档、现有代码	WXML/WXSS/JS 页面、服务层、校验脚本
微信小程序框架	app.json、Page 生命周期、WXML 模	可运行的小程序目录结构

	板规则	
自动校验脚本	路由、页面四件套、Schema、页面原型 覆盖	npm run check 质量门禁



图 1 项目技术结构图

三、功能架构与页面路由

小程序当前注册 17 个页面。底部 TabBar 提供首页、圈子、发布、消息、我的五个主入口；交易、认证、搜索、订单、钱包等页面通过 wx.navigateTo 进入。这样的路由结构保证了课程 Demo 的完整性和主要功能流程的连贯性。

序号	页面路径	入口类型
1	pages/index/index	TabBar
2	pages/community/index	TabBar
3	pages/publish/index	TabBar
4	pages/message/index	TabBar
5	pages/message/chat	二级页面

6	pages/profile/index	TabBar
7	pages/auth/login	二级页面
8	pages/goods/detail	二级页面
9	pages/goods/offer	二级页面
10	pages/order/checkout	二级页面
11	pages/order/detail	二级页面
12	pages/search/filter	二级页面
13	pages/favorites/index	二级页面
14	pages/wallet/index	二级页面
15	pages/safety/index	二级页面
16	pages/seller/store	二级页面
17	pages/publish/success	二级页面

主要用户路径为：首页浏览商品 -> 商品详情 -> 出价或立即购买 -> 结算页 -> Demo 支付自动通过 -> 订单详情 -> Demo 核销自动完成。未认证用户在发布和支付节点会被导向学生认证页。

四、数据模型与本地服务层

核心数据模型关系图

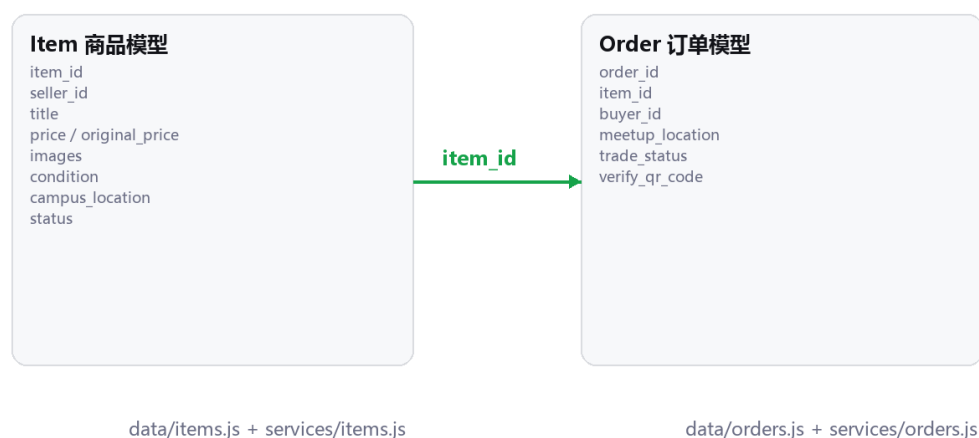


图 2 Item 与 Order 核心数据模型

数据层保留课程需求要求的 Item 和 Order 字段，不在 seed 数据中随意扩展字段。为了让 Demo 具备完整功能结构，services 目录新增本地业务服务：auth 负责认证状态，items 负责商品列表与发布，orders 负责模拟交易状态机，chats 负责会话和消息追加。

模块	文件	职责
商品数据	data/items.js	提供 Item seed 数据与 getItems / getItemById
订单数据	data/orders.js	提供 Order seed 数据与 getOrders / getOrderById
认证服务	services/auth.js	封装学生认证、认证状态读取、未认证阻断
商品服务	services/items.js	封装搜索、发布 Demo 商品、收藏列表映射
订单服务	services/orders.js	封装 wait_pay / wait_meetup / completed 状态流转
聊天服务	services/chats.js	封装会话列表、聊天记录与发送消息
演示说明	DEMO_GUIDE.md	记录微信开发者工具演示路径、关键文件和 Demo 边界

五、核心业务流程实现

5.1 学生认证与准入控制

认证页采用 login 和 verify 两阶段状态。用户先勾选协议，再进入校园邮箱验证。Demo 版本通过 .edu 邮箱判断认证成功，并写入 app.globalData.isVerified。发布页和结算页不直接判断页面变量，而是调用 requireVerified，保证阻断逻辑集中在服务层。

学生认证与权限控制流程



核心服务: `services/auth.js` 提供 `getAuthState`、`verifyStudent`、`requireVerified` 三个函数，页面只负责调用和导航。

图 3 学生认证与权限控制流程

5.2 Demo 担保交易与核销

课程作业不需要真实支付与真实扫码核销，因此交易流程采用“结构完整、资金模拟”的实现策略。结算页点击确认支付后，服务层先创建 `wait_pay` 订单，再立即调用 `payDemoOrder` 将状态推进到 `wait_meetup`；订单详情页点击确认收货后调用 `completeDemoOrder`，将状态推进到 `completed`。

Demo 交易闭环流程图

期末作业 Demo 不接真实金钱交易：点击支付与核销后由本地服务自动推进状态



状态机: wait_pay -> wait_meetup -> completed / cancelled

实现位置: services/orders.js; 页面入口: pages/order/checkout 与 pages/order/detail

图 4 Demo 交易闭环流程

5.3 页面与服务层的衔接

1. pages/goods/detail 接收 item_id，展示商品，并在出价前调用认证阻断。
2. pages/goods/offer 根据商品价格生成默认九折出价，并把 offer_price 传入结算页。
3. pages/order/checkout 使用出价金额计算费用明细，创建 Demo 订单并模拟支付通过。
4. pages/order/detail 根据订单状态生成时间线，点击确认收货后自动完成核销。
5. pages/message/chat 通过 chat_id 获取会话与商品信息，支持追加本地消息。
6. pages/publish/index 绑定标题和价格输入，已认证用户提交后创建本地 Demo 商品。

六、界面效果与代码证据

以下界面图为根据当前页面结构绘制的初步运行截图式示意，可用于文档初版占位。后续在微信开发者工具完成真机预览后，可以替换为真实模拟器截图。

初步运行界面截图式示意

根据当前 WXML/WXSS 页面结构绘制，用于 Word 初稿展示；最终可在微信开发者工具中替换为真机截图。



图 5 首页、商品详情、结算与订单核销的初步运行截图式示意

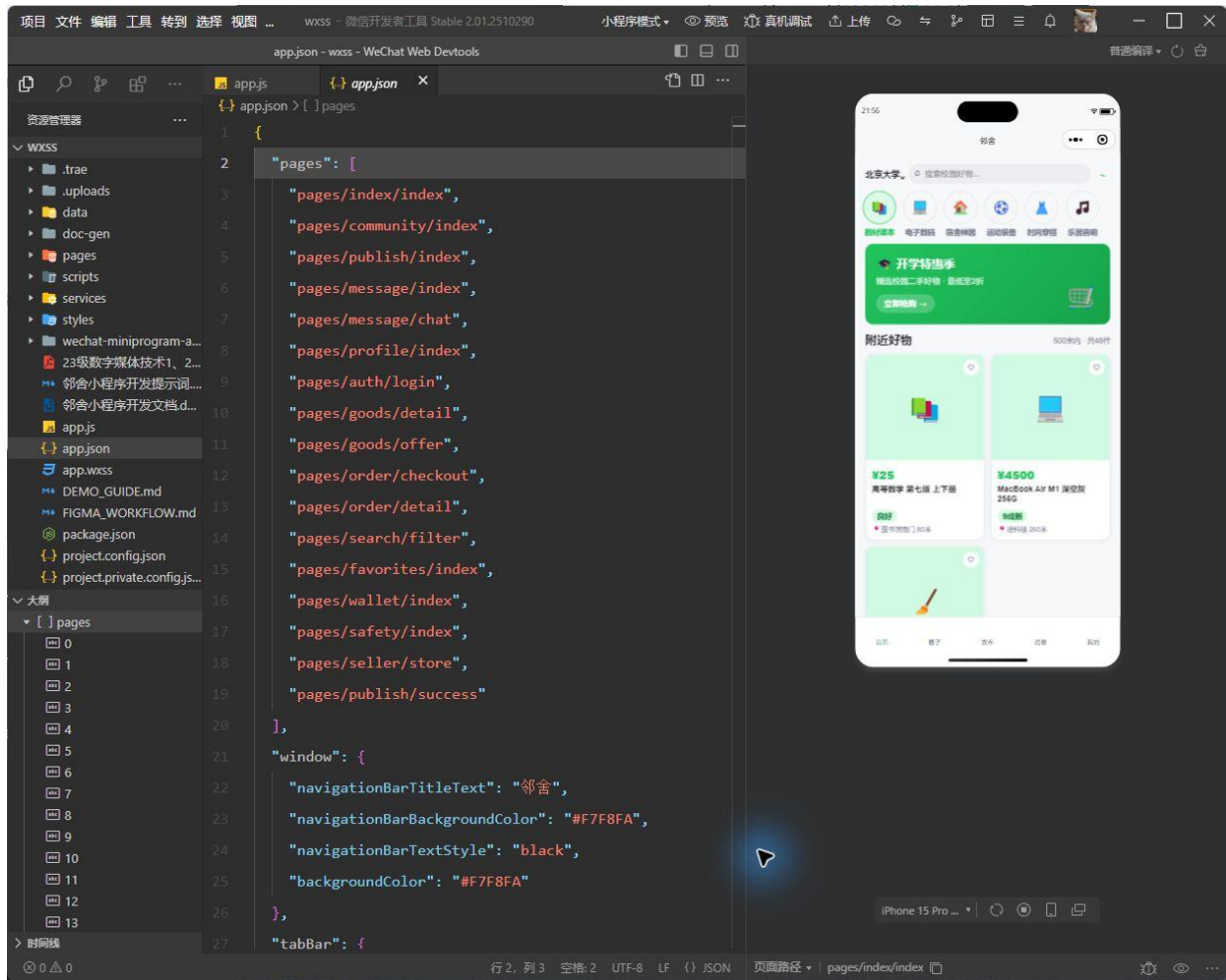


图 6 微信开发者工具 GUI 实际运行截图：首页已在模拟器中显示，问题面板为“没有问题”

核心代码截图覆盖路由配置、认证服务、订单状态机和自动校验脚本四个关键点。这些截图说明项目不是单纯静态页面，而是具备路由、状态、服务封装和质量门禁的完整 Demo 工程。

app.json - 路由与 TabBar 配置

```
01 {
02   "pages": [
03     "pages/index/index",
04     "pages/community/index",
05     "pages/publish/index",
06     "pages/message/index",
07     "pages/message/chat",
08     "pages/profile/index",
09     "pages/auth/login",
10     "pages/goods/detail",
11     "pages/goods/offer",
12     "pages/order/checkout",
13     "pages/order/detail",
14     "pages/search/filter",
15     "pages/favorites/index",
16     "pages/wallet/index",
17     "pages/safety/index",
18     "pages/seller/store",
19     "pages/publish/success"
20   ],
21   "window": {
22     "navigationBarTitleText": "🏠",
23     "navigationBarBackgroundColor": "#F7F8FA",
24     "navigationBarTextStyle": "black",
25     "backgroundColor": "#F7F8FA"
26   },
27   "tabBar": {
28     "color": "#6B6E85",
29     "selectedColor": "#22C55E",
30     "backgroundColor": "#FFFFFF",
31     "borderStyle": "black",
32     "list": [
33       {
34         "pagePath": "pages/index/index",
```

图 7 app.json 路由与 TabBar 配置代码截图

```
services/orders.js - Demo 订单状态机
01  const { getOrders, getOrderById } = require("../data/orders");
02  const { getItemById } = require("../items");
03
04  function getDemoQr(orderId) {
05    return `demo-verify-code:${orderId}`;
06  }
07
08  function createDemoOrder(payload) {
09    const item = getItemById(payload.item_id || "item_003");
10    const order = {
11      order_id: `order_demo_${Date.now()}`,
12      item_id: item.item_id,
13      buyer_id: payload.buyer_id || "buyer_001",
14      meetup_location: payload.meetup_location || item.campus_location,
15      trade_status: "wait_pay",
16      verify_qr_code: getDemoQr(item.item_id)
17    };
18
19    getOrders().unshift(order);
20    return order;
21  }
22
23  function payDemoOrder(orderId) {
24    const order = getOrderById(orderId);
25    order.trade_status = "wait_meetup";
26    order.verify_qr_code = getDemoQr(order.order_id);
27    return order;
28  }
29
30  function completeDemoOrder(orderId) {
31    const order = getOrderById(orderId);
32    order.trade_status = "completed";
33    return order;
34  }
```

图 8 Demo 订单状态机代码截图

七、质量校验与测试结论

本项目使用 npm run check 作为基础质量门禁。校验命令覆盖文件存在性、JSON 可解析性、中文配置、TabBar 文案、所有页面四件套、WXML 事件处理函数、页面跳转路由、Item/Order 数据结构，以及服务层的认证、订单状态机和聊天追加能力。Item / Order 会检查字段集合完全一致，并验证 condition、status、trade_status 枚举。校验过程还会加载 app.js 与全部页面 JS，并执行主流程 smoke test，模拟认证、出价、结算金额、支付、核销、聊天、发布、发帖和反馈等演示路径。

检查项	校验方式	结果
页面完整性	遍历 app.json 中 17 个页面，检查 .json/.js/.wxml/.wxss	通过
配置正确性	解析 app.json / project.config.json / sitemap.json	通过

中文可读性	检查标题“邻舍”和 TabBar 文案	通过
数据模型	逐项检查 Item 9 字段与 Order 6 字段	通过
枚举约束	检查 Item.condition/status 与 Order.trade_status 合法值	通过
服务层流程	模拟认证、支付、核销、聊天追加	通过
JS 加载检查	注入微信全局对象桩并加载 app.js 与全部页面 JS	通过
主流程 smoke test	模拟认证、出价、Demo 支付、订单核销、聊天发送、发帖和反馈	通过
页面原型 对齐记录	检查 18 个 页面原型 名称是否写入 页面对齐记录	通过

最近一次执行结果: Lin She mini program check passed。

微信开发者工具 GUI 检查结果: 已打开 wxss 项目, 模拟器显示 pages/index/index 首页, 问题面板显示“没有问题”。

八、当前不足与后续计划

当前版本已经适合作为期末作业 Demo 的初版基础, 但距离可上线产品还有明显差距。后续工作应按课程提交优先级推进: 先补真实运行截图和答辩演示路径, 再考虑云开发、图片上传与真实支付等非 Demo 能力。

不足	原因	后续处理
运行截图仍为初步示意	当前未在微信开发者工具中逐页截屏	用模拟器截图替换图 5 或补充更多页面截图
数据为本地内存	课程 Demo 优先展示结构与流程	接入 CloudBase 数据库
支付核销为模拟	作业不涉及真实金钱交易	正式产品需接微信支付与后台核销
图片上传为占位	当前重点是页面与流程	接入 wx.chooseImage / wx.uploadFile
聊天非实时通讯	当前为本地消息数组	后续接 WebSocket 或云开发实时数据库

九、参考资料

- 微信小程序官方文档: <https://developers.weixin.qq.com/miniprogram/dev/framework/>
- 微信小程序全局配置 app.json:
<https://developers.weixin.qq.com/miniprogram/dev/reference/configuration/app.html>
- 微信小程序 WXML 语法参考:
<https://developers.weixin.qq.com/miniprogram/dev/reference/wxml/>
- 微信小程序 Page 构造器与生命周期:
<https://developers.weixin.qq.com/miniprogram/dev/reference/api/Page.html>
- 微信小程序官方开发文档与课程项目开发资料
- 项目内运行说明、页面结构记录与项目校验命令

十、正式提交前检查

正式提交前对项目进行了系统 Debug 与交付材料清理：保留开发步骤、功能实现和问题解决记录，去除仅用于开发阶段的辅助说明；重新检查首页商品数据、分类覆盖、页面跳转、按钮交互和订单状态流转，确认 Demo 操作链路完整。

本次整理同时生成 Word 与 PDF 两份开发文档，并以压缩包形式提交完整小程序源码和配套文档。源码目录不包含临时输出、版本控制目录、开发过程记录或无关依赖目录。